Première année du cycle ingénieur - Sup Galilée

Spécialité :Informatique

Calculateur d'itinéraire pour le métro parisien

Projet de programmation en langage C

Réalisé par :

- Imene MOUSSAOUI 12309982
- Anissa HADJOU 12309147

Encadré par :

M. John Chaussard

Année universitaire 2025–2026

Engagement de non-plagiat

Nous, soussignées **Imene MOUSSAOUI** ,**Anissa HADJOU**, étudiantes en première année d'école d'ingénieur à **Sup Galilée**, déclarons être pleinement conscientes que la copie de tout ou partie d'un document, quel qu'il soit, publié sur tout support existant, y compris sur Internet, constitue une violation du droit d'auteur ainsi qu'une fraude caractérisée, tout comme l'utilisation d'outils d'Intelligence Artificielle pour générer une partie de ce rapport ou du code associé.

En conséquence, nous déclarons que ce travail ne comporte aucun plagiat, et assurons avoir cité explicitement, à chaque fois que nous en avons fait usage, toutes les sources utilisées pour le rédiger.

Fait à Villetaneuse , le 06/02/2026

Signatures :

Imene MOUSSAOUI **Anissa HADJOU**

Sommaire

1 Introduction **4**

2 Modélisation du problème **4**
2.1 Les structures de données 4
2.2 Correspondances 5

3 Algorithme de Dijkstra **7**

4 Gestion mémoire **7**

5 Compilation **8**

6 Conclusion **8**

7 Sources **9**

1 Introduction

L'objectif de ce projet est d'implémenter en *Langege C* un calculateur d'itinéraire pour le métro ,à partir d'une station de départ et une station d'arrivée le programme doit trouver le chemin le plus court entre ces deux stations en tenant compte du temps de parcours entre stations et le temps nécessaire pour un changement de ligne.
Le projet repose sur la modélisation du réseau du métro sous forme d'un graphe orienté pondéré,et sur l'application de l'algorithme de *Dijkstra* ,construit à partir de deux fichies CSV.

2 Modélisation du problème

Dans notre graphe, les sommets représentent les stations de métro, chaque sommet a son unique identifiant et la ligne du métro correspondante tandis que les arêtes représentent les différentes connexions entre les stations (sommets).

2.1 Les structures de données

Structure Station:

```
typedef struct station{
    int id;
    char *nom;
    struct station *suivant_st;
}station;
```

Cette structure permet de créer des stations avec un identifiant unique et un nom de station; qui sera ensuite utilisée pour créer une liste chaînée de stations, permettant de stocker toutes les stations lues depuis un fichier CSV.

Structure liste de stations :

```
typedef struct list_station{
    station *tete;
    int taille;
}list_station;
```

Cette liste permet d'afficher à l'utilisateur toutes les stations avec leur identifiant.

Structure Sommet:

```
typedef struct sommet{
    station *parada;
    char* ligne;
    struct sommet *suivant;
}sommet;
```

Pour les sommets du graphe , c'est une structure qui contient un pointeur vers une station (id,nom),la ligne du métro associée et un pointeur vers un autre sommet dans la liste des sommets du graphe.
Deux sommets distincts peuvent représenter la même station mais avec sur des lignes différentes.

Structure Arête:

```
typedef struct arete{
    int cout;
    int id_destination;
    struct arete *next;
}arete;
```

La structure arête permet de créer une connexion entre les stations en calculant le "coût" (le temps du trajet) jusqu'à la station d'arrivée dont l'identifiant est "id_destination".L'utilisation d'une liste chaînée a pour avantage de stocker un nombre variables d'arêtes pour chaque sommet.

Structure Graphe:

```
typedef struct graph{
    int nb_sommet ;
    sommet *list_sommet;
    arete **list_arete;
}graph;
```

Cette structure contient le nombre total des sommets, une liste chaînée des sommets (sommet *list_sommet) et un tableau indexé(arete **list_arete) telle que chaque case i correspond à un pointeur vers une liste d'arête d'un sommet 'v' d'index i dans la liste des sommets.Cette modélisation offre un accès rapide aux arêtes d'un sommet tout en permettant une gestion dynamique des sommets.

2.2 Correspondances

Lecture et ajout d'arêtes:

La fonction **lire_aretes** lis les connexionsdepuis le fichier *"Metro Paris Data - Aretes.csv"* et construit le graphe complet représentant le réseau métro.Avec la fonction:

```
int exist_sommet(graph *g,int id, char *ligne);
```

On vérifie avant d'ajouter une connexion si un sommet existe déjà dans le graphe, cette fonction parcourt la liste des sommets du graphe, retourne l'index si sommet trouvé sinon -1.
Si exist_sommet retourne -1, cela signifie que la station n'est pas encore dans le graphe pour cette ligne:

- On récupère le nom de la station dans la liste générale des stations (list_station) grâce à find_name.
- On crée un nouveau sommet avec ajouter_sommet, qui :
- Ajoute la station à la liste des sommets du graphe.
- Met à jour dynamiquement le tableau list_arete pour y stocker ses arêtes.
- Chaque sommet est donc unique pour un couple (id, ligne).

Une fois les sommets existants ou nouvellement crées,on ajoute les arêtes tel que :

- Une arete ajoutée entre la source et la destination.
- Si les sommets appartiennent à la même ligne, le coût de l'arête est égale à 1.
- Si les sommets représentent la même station mais sur des lignes différentes, une correspondance est créée avec un coût qui es égale à 5.

Cette logique est implémentée dans la boucle ci-dessous:

```
int index_src=exist_sommet(g,src,ligne_metro);
int index_des=exist_sommet(g,dest,ligne_metro);
   if(index_des==-1){
       char *name=find_name(list_st,dest);
       if(!name) {
        printf("Erreur : station id %d non trouvée\n", dest);
        continue;
       }
       ajouter_sommet(g,dest,name,ligne_metro);
       free(name);
       index_des=g->nb_sommet-1;
       }
   if(index_src==-1){
       char *name=find_name(list_st,src);
       if(!name) {
       printf("Erreur : station id %d non trouvée\n", dest);
       free(name);
       continue;
}
       ajouter_sommet(g,src,name,ligne_metro);
       free(name);
       ajouter_arete(&(g->list_arete[g->nb_sommet-1]),index_des,1);
   }else{
       ajouter_arete(&(g->list_arete[index_src]),index_des,1);
   }
   }
```

```
sommet *s1=g->list_sommet;
for(int i=0;i<g->nb_sommet;i++){
   sommet *s2=s1->suivant;
   for(int j=i+1;j<g->nb_sommet;j++){
       if(s1->parada->id==s2->parada->id){
           ajouter_arete(&(g->list_arete[i]),j,5);
           ajouter_arete(&(g->list_arete[j]),i,5);
       }
       s2=s2->suivant;
   }
   s1=s1->suivant;
}
```

Ainsi le graphe contient les stations comme sommets différenciées par ligne,les connexions directes sur la même ligne et les correspondances entre les lignes sur la même stations avec un coût égale à 5.

3 Algorithme de Dijkstra

L'algorithme de Dijksra à été choisi pour le calcul du plus court chemin dans le graphe réseau de métro ,car notre réseau est représenté par un graphe où les sommets correspondent aux stations et les arêtes ont un poids positif qui représente le temps du trajet. L'algorithme de Dijksra facilite l'ajout de nouveaux sommets ou d'arêtes sans nécessiter de modification algorithmiques.

```
void dijkstra(graph *g, int src, int dest);
```

Cette fonction utilise :

- dist[i]:pour la distance minimale connue entre la source et i.
- pred [i]:sommet précedent sur le chemin le plus court.
- visited[i]:indique si le sommet i à été visité.

A chaque étape ,on sélectionne le sommet non visité avec la distance minimale et on met à jour les distances de ses voisins.Et puis l'affichage avec la fonction:

```
void afficher_chemin(int *prev, int src, int dest, graph *g);
```

4 Gestion mémoire

Toutes les structures dynamique sont correctement libérées, de plus, la validité de cette gestion mémoire a été vérifiée à l'aide de l'outil Valgrind.

```
void free_station(station *st);
void free_list_station(list_station *list);
void free_sommet(sommet *s);
void free_arete(arete *ar);
void free_graph(graph *graph);
```

5 Compilation

Pour compiler, nous avons créé un fichier Makefile, afin de faciliter la compilation en entrant la commande ***make***

6 Conclusion

Ce projet nous a permis d'appliquer concrètement les notions théoriques vues dans le module *Algorithmes dans les graphes*,,en les appliquant à un cas réel : la modélisation d'un réseau de métro.La représentation de sous réseau sous forme de graphe en utilisant l'algorithme de Dijkstra à permis de résoudre efficacement du problème du plus court chemin.

D'un point vu technique, ce projet a renforçé nos compétences en langage C, notamment ce qui concerne la manipulation des structures de données, les listes chaînées, la gestions de la mémoire en utilisant *Valgrind* et le traitement des chaînes de caractères.

A travers ce projet, nous avons appris à analyser un problème complexe, à choisir les structures de données adaptées et à trouver une solution algorithmique efficace et cohérente.

7 Sources

Chatgpt: Pour ajouter des couleurs au menu.